

Vasic-Digital-Reusable-Module-Suite

一次构建，随处复用——一组小型、解耦、独立测试的**Go**与**KMP**模块

概述

一系列通用、可复用的模块，发布于digital.vasic.* (Go) 及Kotlin Multiplatform命名空间下。每个模块均独立存在，拥有独立的测试与版本控制，并作为同等代码库的子模块被更大型产品（Catalogizer、HelixAgent及更广泛的模块群）所调用。本页面将众多小型工具整合于此，避免其作为独立页面显得过于琐碎。

简要说明

一套精心设计的解耦digital.vasic.*模块集合，涵盖基础设施基元（认证、缓存、数据库、配置、可观测性）、AI/代理构建模块（RAG、向量数据库、嵌入、MCP、智能体、规划）及防御性LLM防护机制（红队测试、标准化处理），同时包含Kotlin Multiplatform镜像模块集。每个模块均通用、经过测试且可复用。

详细说明

vasic-digital组织的运作基于一项结构性抉择：采用“宪章+众多解耦可复用子模块”的理念，确保通用功能绝不重复开发。与单体架构不同，每个可复用组件均被提取为独立模块——拥有独立的代码库、测试及文档——并严格保持解耦，确保任何使用方的特定需求不会渗入其中。本页面将这些模块汇总，因单独来看，每个模块规模仅相当于一个库，若作为独立产品页面则显得过于零散；而整体来看，它们构成了组织真正的效率倍增器：一项私有工程资产，将“构建新产品”转化为“组装成熟组件”，并为“本模块群不重复造轮子”的承诺提供了切实支撑——它维护的是一个极为优秀的“轮子”，并将其应用于各处。

该模块套件涵盖三大类别。基础设施基元（Go）提供各类服务所需的基础组件：auth (JWT/bcrypt)、cache (Redis/TTL)、database (迁移、双SQLite/PostgreSQL)、config、middleware、observability (Prometheus/OpenTelemetry)、ratelimiter、security、storage (S3/MinIO)、streaming (WebSocket中心)、eventbus、filesystem

(多协议)、discovery/mdns、http3、recovery、concurrency、lazy等。**AI/代理构建模块** (Go) 为AI系统提供基础支撑: rag、vectordb、embeddings、memory、conversation (无限上下文压缩、事件溯源)、mcp (Model Context Protocol)、toolschema、skillregistry、agentic (基于图的工作流编排)、planning (HiPlan/MCTS/思维树)、benchmark (SWE-bench/HumanEval/MMLU)、llmops、selfimprove (奖励建模/RLHF) 及toon (面向Token的对象表示法)。防御性**LLM**防护机制提供对抗鲁棒性工具: RedTeam (基于YAML的对抗性测试用例)、Normalize (对抗性输入规范化)。此外, **Kotlin Multiplatform**模块集还提供核心模块的跨平台镜像版本 (Auth-KMP、Database-KMP、Security-KMP、UI-Components-KMP等)。

内容

我们为何构建它

每次从零开始开发多个产品 (Catalogizer、HelixAgent、Herald等) 既浪费资源, 又难以保持一致性。将所有通用功能抽取为解耦且经过验证的模块后, 修复和改进能够自动传递至整个产品线, 而每个新产品则由经过验证的组件拼装而成。

为何它是颠覆性的

这实际上是为构建以AI为核心的后端系统打造的私有“标准库”——大多数团队无暇构建这一层, 因为他们总在第四五次重新解决身份验证、缓存和RAG基础设施等问题。在这里, 基础设施原语、AI构建模块以及防御性LLM防护机制均以即插即用、独立测试的模块形式存在。这使得小团队能以通常需要更大规模团队才能达到的速度交付产品级系统, 且不会在过程中积累重复性技术债务。

创新之处

- 全产品线解耦规范 (**CONST-051**): 子模块视为独立代码库, 绝不包含消费者特定逻辑。
- 专用AI原语层 (**RAG**、向量数据库、嵌入模型、**MCP**、工具模式、智能体、规划、**LLMOps**): 以可复用模块形式呈现。
- 防御性**LLM**防护集群 (**RedTeam**、**Normalize**): 用于对抗性鲁棒性测试。
- 并行**Go + Kotlin Multiplatform**模块集: 共享相同设计规范。

挑战与解决方案

- 避免数十个模块间的耦合腐化: 通过宪章解耦协议及运行时注入消费者特定逻辑解决。

- 保持多模块的一致性与可测试性：通过共享约定（每模块包含测试/文档/挑战）及HelixConstitution治理框架解决。
- 跨平台覆盖：通过Kotlin Multiplatform核心模块镜像解决。

技术栈（选择原因及实现方式）

- **Go** —— 大部分模块（digital.vasic.*）。
- **Kotlin Multiplatform** —— 跨平台镜像模块（身份验证/数据库/安全/UI/并发/限流器-KMP）。
- **Redis / PostgreSQL / SQLite** —— 缓存、数据库、存储原语。
- **Prometheus / OpenTelemetry** —— 可观测性模块。
- **WebSocket / HTTP/3 (quic-go) / mDNS** —— 网络模块。
- **向量数据库 / embeddings / RAG / MCP** —— AI原语模块。
- **YAML** —— RedTeam对抗性测试用例及配置。

未验证/开发中：多个组织仓库自标记为“脚手架/开发中”（如PliniusCommon、I-LLM、HyperTune、AutoTemp、Veritas、Ouroborous、Claritas、LeakHub、GandalfSolutions）。这些内容应视为早期阶段/脚手架，尚未正式发布。